

Sight Reading Pro

Final Project Report - X words

Theresa Thoraldson

2026-09-28

Contents

Introduction - X words	3
An overview	3
Why Would People Want This Project?	4
Existing/Similar Projects	4
Literature Review - X words	6
Exploring Sight-Reading in Education	6
Pitch Estimation, Tempo Estimation, Audio Evaluation	6
Music Generation	7
The Research Gaps	8
Design - X words	8
Domain and Project Users	8
Design Overview	8
Design Choices	12
User Feedback	12
Implementation - X words	12
Overview	12
Docker	12
Frontend	13
Main Backend API	13
Model APIs	13
Evaluation - X words	13
Model Evaluation	13
Failed Approaches	14
Successes	14
Limitations	14
Extensions	14

Conclusion - X words	14
In Conclusion...	14
If I were to do it again	14
Further Work	15
Acknowledgements	16
References	16

This project code be found here: <https://github.com/tthoraldson/uol-final-project/>

Introduction - X words

The template I am choosing is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal.

In music, there is a concept called sight-reading. It involves being given a piece of sheet music and trying to play or sing it right away with little or no preparation. According to the National Association for Music Education, the ability to sight-read music has many benefits including increased confidence, stronger foundations in rhythm and pitch, and less stress when it comes to learning new music pieces [3].

An overview

“Sight Reading Pro”, the name of this project, will be an application where users can practice and master sight-reading. Users will be able to use any instrument of their choosing, including voice, to practice their sight-reading abilities.

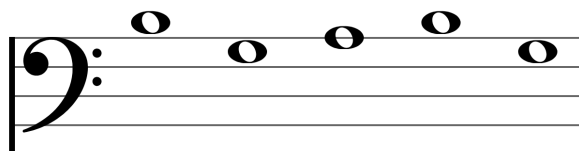


Figure 1: Example sight-reading passage

Sight Reading Pro will be a web based application, utilizing various state of the art machine learning models to generate music passages for users to practice their sight-reading abilities. The user will get immediate feedback on attempts, including intonation (did they play the correct note?) and rhythm (did they play at the correct time?).

Users can choose whether they want to have an account or not. If they choose to have an account, they can keep track of all of the passages they’ve generated, and their recorded attempts and feedback on each passage.

Users with or without a login will be able to customize the complexity and difficulty of the generated passages using settings. Users will also be able to select between different styles of passages, such as classical or jazz.

Why Would People Want This Project?

Personally, I've always wanted to get better at sight-reading but haven't found any existing products on the market that make me stick with working through exercises. The generated exercises are often boring, and only target one music area at a time (i.e. advance rhythms, chords, harmony, etc). Having the ability to customize the exercises, and adjusting them to meet my current music learning goals would be amazing.

The existing free projects on the market have limited instrument support, and have no way to track exercise history. The paid options are more focused on music education, instead of individual learning goals. The paid options are also out of reach to users that want to learn sight-reading, but don't have the budget to do so.

My goals for the project are to make a free, open source, individualized sight-reading platform that can work with any instrument.

Existing/Similar Projects

Sight Reading Factory

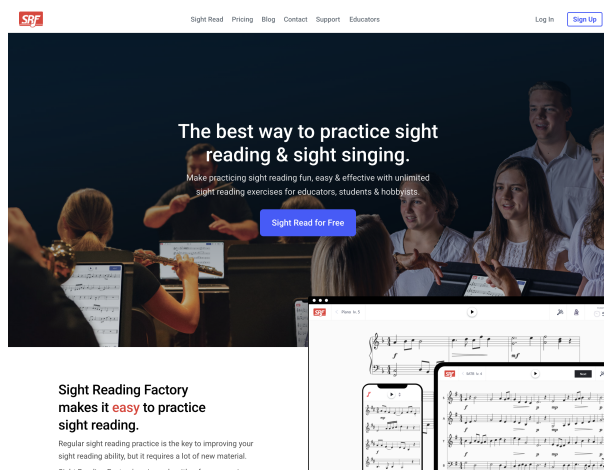


Figure 2: Sight Reading Factory screen capture

Sight Reading Factory[11] is a platform for sight-reading exercises mainly aimed towards educators. This platform supports ~30 instruments, and offers different difficulties of music passages to play. They offer generated music exercises for learners to practice with, getting feedback with each attempt at playing a passage.

Advantages Sight Reading factory has a wide variety of features, including the ability to assess how well a user did on a given passage of music, and the

ability to control how difficult generated passages are by limiting the note range, tempo, and rhythmic complexity. Sight Reading Factory supports both general audio/microphone input and MIDI.

Sight reading factory allows you to keep track of what passages you have generated, all of the sight-reading attempts you have made on a given passage, and it offers assessments on new blind passages. This product is the most robust with features that I have found.

Disadvantages I believe the biggest disadvantage to Sight Reading Factory is the fact that it's a subscription, and starts at \$45 per user per year. There's a free trial, but you have to enter credit card information in order to try it.

Sight Reading factory supports 30 instruments, but has no generic option if your instrument is not supported. You must log in to do anything, which can be just enough friction for learners that want to try sight -eading without committing to a product. This product is also heavily influenced by music educators, and a lot of the features are what you would expect to be in a classroom environment. For example, getting assigned passages to work on.

sightreading.training

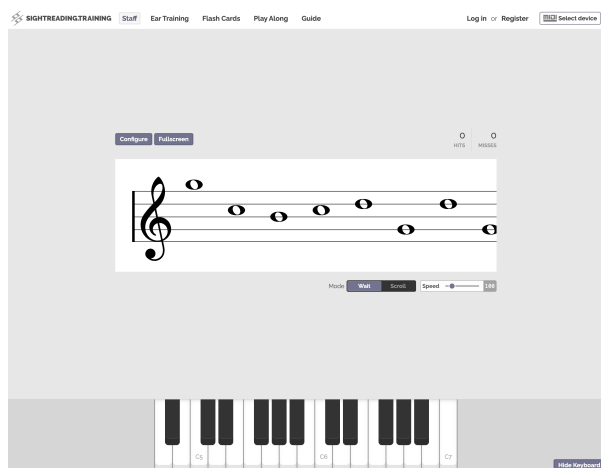


Figure 3: sightreading.training screen capture

sightreading.training[12] is a free and open source sight-reading platform. It offers sight-reading tools, and other tools for learning and playing music.

Advantages sightreading.training supports MIDI, or allows the user to play on a piano that's in the browser. Having a virtual piano is a cool idea, as it makes sight-reading accessible to everyone, including those that don't have an instrument of their own. This website is also free, and you can get started

practicing right away. Users have the ability to login and save settings for their sight-reading passages, but not the passages themselves.

Disadvantages sightreading.training only supports MIDI and the virtual piano. If you have an instrument that you can't plug into your computer, you can't use this application. There are no settings to change rhythmic complexity. All of the notes have the same length.

Literature Review - X words

Exploring Sight-Reading in Education

Understanding sight-reading, and how to improve this ability has been an area of study for over 100 years. One research study shows that aural-spatial skills (ear training) and technical proficiency skills were both essential to sight-reading[21].

In more recent times, a review was done on AI-education research related to music education. It concludes that "AI is under-utilized for generation despite its potential for education"[19]. The music generation part of the literature review goes deeper into music generation related to music exercises for general technical improvement.

Pitch Estimation, Tempo Estimation, Audio Evaluation

There are lots of existing pre-trained models available that do different tasks related to music. This is an overview of the models I found most relevant to this project, and the models that I will test for usage in this project.

CREPE

CREPE is a deep convolutional neural network that does pitch estimation [22]. Given an audio file, such as a .wav or .mp3 file, CREPE will give a predicted frequency in hertz alongside a confidence score for every 10 milliseconds. Having a confidence interval allows for

There is also a demo of CREPE[1] that runs fully in the browser using Tensorflow JS (tfjs)[14]. It shows real time pitch estimation based on microphone input. In regards to this project, being able to have real time pitch estimation would allow for immediate feedback on a music passage.

TempoCNN

TempoCNN[24] is a group of Convolutional Neural Network (CNN) models that estimate a given audio file's tempo, measured in beats per minute (BPM). These modes were trained on datasets that included mainly ballroom dancing music, and electronic dance music. This paper does note that there is a lack of various

genres of music, including jazz, classical or reggae, and believes that the models would perform better if they had found and included this kind of data.

For my project “Sight Reader Pro”, I believe TempoCNN is a good candidate model for being able to automatically detect the tempo that the user’s recording was played at, and also use it for giving feedback on a given passage.

Music Generation

Basic Pitch

Basic Pitch[18] is a model built by spotify that takes in an audio file (.wav or .mp3), and returns a MIDI file containing either the single pitch (one instrument/voice) or multi pitch (multiple instruments/voices/notes). Basic Pitch works in the realm of “Automatic Music Transcription”, an area of research used to automatically create symbolic representations of music.

I believe Basic Pitch is a good candidate model that I can use to create backing tracks, and be able to have them stored as midi files. This way the project can take advantage of technologies such as WebMIDI[16] for playback of a generated passage.

MusicGEN

In “Simple and Controllable Music Generation”, Meta AI describes a Language Model called MusicGEN[20]. MusicGEN is capable of taking a text based input, and generating a full song. This model can also be prompted with an existing melody and a text prompt, and output a song based on the original melody.

This series of models is promising for offering customized backing tracks for generated sight-reading exercises. The exercises could be tailored to include the correct genre of backing track.

While the music generated is really high quality, the models are also large and require a lot of compute. The smallest model in the MusicGEN model series contains 300 million parameters, and requires ~16gb of GPU RAM to run.

NotaGen

NotaGen[27] is a model that generates classical sheet music. The paper proposes a new “ClaMP-DPO” method for reinforcement learning, which increases musicality when compared to traditional human annotation or predefined rewards.

This model requires at least 8GB of GPU RAM to run the smallest model. One goal of “Sight Reader Pro” is to have low response times for any generative process. Because of the large compute requirements for NotaGEN, I don’t believe it will be a good fit for the project.

Text To Music

Chat Musician

Text2Midi

The Research Gaps

While there are

Design - X words

As stated in the intro, the template I am choosing is Artificial Intelligence Project Template 1: Orchestrating AI Models to Achieve a Goal.

Domain and Project Users

The goal of this project is to create a sight-reading web application that utilizes various machine learning models to create unique sight-reading exercises. The *target user* for this project is an independent musician that's looking to improve their sight-reading abilities, and is interested in a customized experience based on their instrument of choice, skill level and music genre of choice. The *domain* of this project could be considered to be music education, but I'm aiming for it to be specifically independent music education.

This project is not ideal for users just learning an instrument, as this application requires at least an elementary understanding of reading sheet music, and playing their instrument of choice.

Design Overview

The designs below are what an minimum viable product for this project will look like. They have all of the bare minimum functionality, and nothing more.

Home / Landing Page

The home page will be a simple call to action to start practicing right now. The user can pick from generating another random exercise, or practice the one displayed right now. There will be no option to customize exercises from the home screen. If "practice now" is clicked, they will be taken to the main practice screen.

Before Recording Exercise Attempt

The exercise page before an attempt is very clean, only including two options: connect to an audio source and record an attempt. If the user selects the record, the record button will switch to a stop button. After the stop button is selected, the screen will update to the "after recording exercise attempt" screen. If the

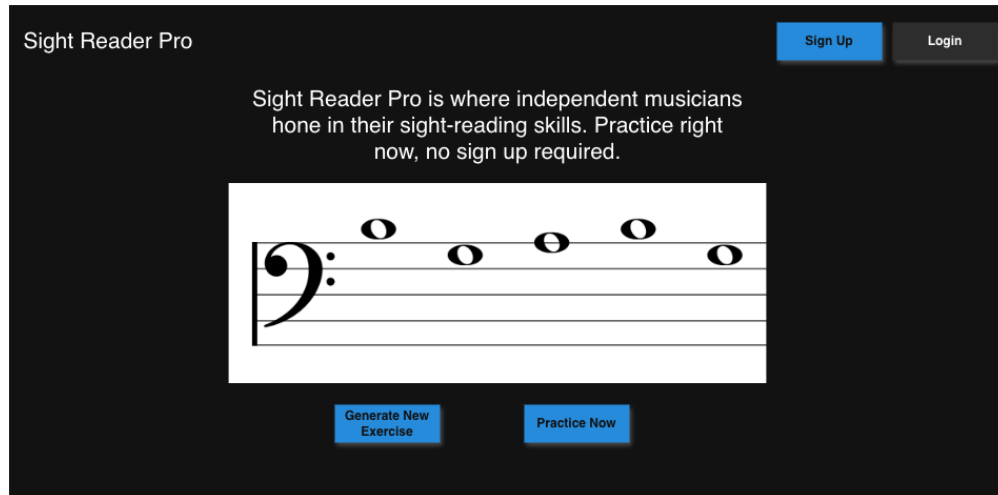


Figure 4: Sight Reading Pro Home Mockup

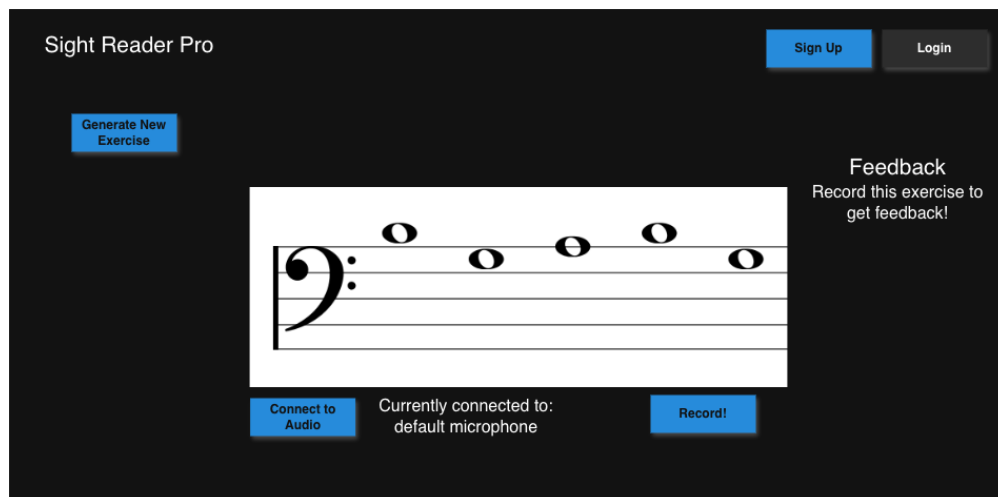


Figure 5: Sight Reading Pro Exercise Practice Before Attempt

user doesn't have an audio source selected, they will not be able to record an attempt.

After I conduct more user interviews I will determine if there is a need for an audio upload option.

After Recording Exercise Attempt

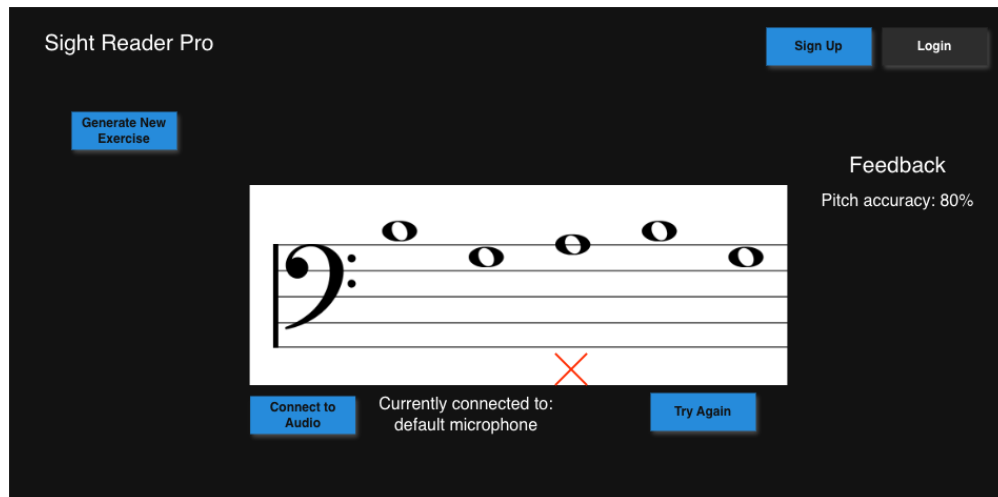


Figure 6: Sight Reading Pro Exercise Practice After Attempt

This screen is extremely similar to the before exercise attempt screen, but with feedback added from the previous attempt. More feedback will be included based on settings selected, including tempo, rhythm accuracy, etc.

Generate New Exercises

In the mockup I have two settings to choose from, a difficulty slider and a genre selector. There will be more settings that I did not include in the mockup, such as:

- Tempo
- Key Signatures
- Melody Complexity (note type selectors)

Exercise History

This mockup shows the Exercise History screen. This screen has all of the exercises that the user has ever generated. The user has the ability to go to an exercise, or to delete it from the list. In this mockup I'm using cards to display each exercise that has been generated.

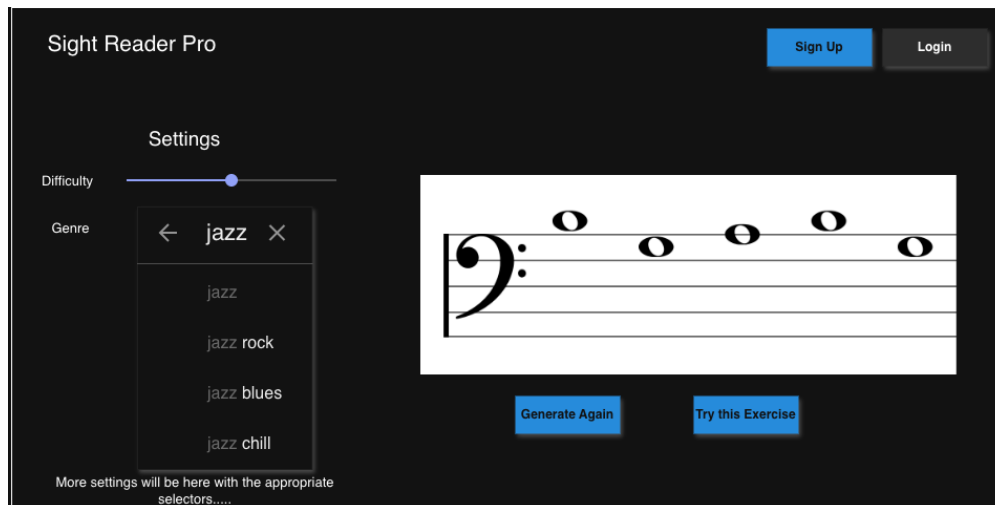


Figure 7: Sight Reading Pro New Exercise

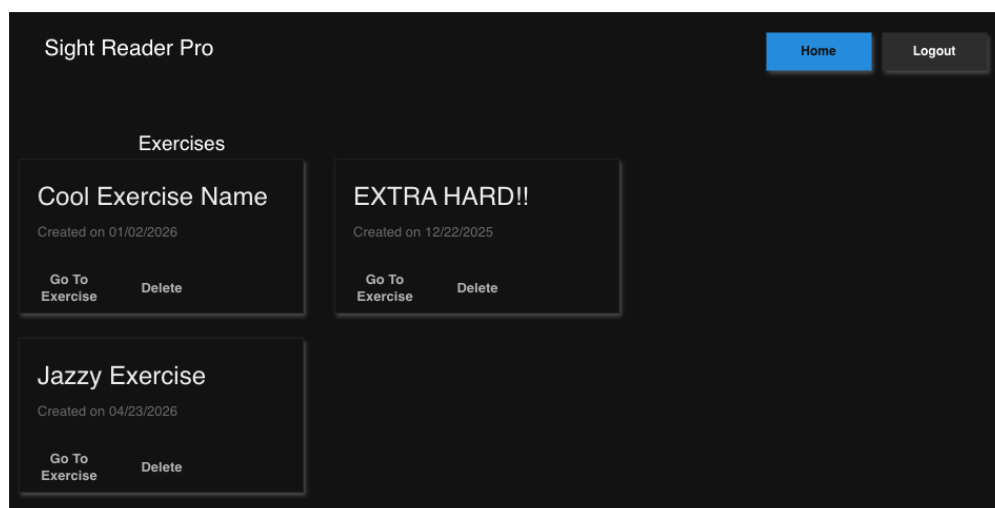


Figure 8: Sight Reading Pro User Exercise History

Design Choices

User Feedback

Good experience with or without an account

My main goal is to reduce friction in users trying out Sight Reader Pro. All of the core functionality will be in the exercise page if a user is logged in or if they're anonymous.

Customizable Exercise Settings

The settings to generate sight-reading exercises will allow users to generate exercises that are specific to them. For example, I only would want to see bass clef exercises that have a jazzy theme. Each user will be able to “choose their own adventure” for what kind of sight-reading experience they want.

Implementation - X words

Overview

Docker

The whole project is wrapped in docker[2]. The frontend, the main API, all of the models, and MLFlow[5] are all in their own containers.

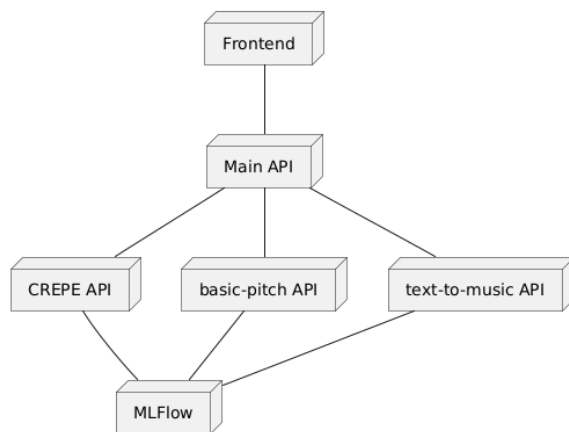


Figure 9: Docker Architecture

There are two defined networks: `frontend-network` and `model-network`. `frontend-network` has the frontend container and the Main API container. The goal of this network was to isolate what could talk to the Main API, and the Frontend. I didn't want any of the model containers being able to communicate

to the frontend, and I didn't want the frontend or any external requests to be able to communicate with the model containers, or the MLFlow container.

Using docker allowed me to better control the dependencies for each model. For example, the `text-to-music model` uses python 3.12, whereas `basic-pitch model` uses python 3.10. These projects would not be able to run in the same container, as they both require different versions of Numpy[6].

Frontend

The frontend was written using React[8] and uses Vite[15] for the docker/production build. I was already familiar with React, as I use it at my day job, and thought it would be the easiest way for me to make the frontend quickly.. The main styling and core components were built using React Bootstrap[9].

Main Backend API

Model APIs

All of the model containers follow the same basic flow, using a version of python that meets the main package requirements, FastAPI for calling the model, and MLFlow for logging each experiment.

Basic-Pitch API

Text-To-Music API

CREPE API

User feedback

Evaluation - X words

Model Evaluation

One of the core parts of choosing this project template was model evaluation. My main goals for choosing models were the following:

- Does this model have fast inference?

MusicGen

- Kinda sucked. Took Forever.

NotaGen

- Also kind of sucked.

Text2Midi

- This model took forever.
- The model took ~2 hours to generate anything worth while.

Text To Music

- Loved the results of this model
- Example code found on Huggingface hub/repo worked

Failed Approaches

Porting TextToMusic to ONNX Runtime

Earlier on in this project, I was playing around with the idea of using Transformers.js[4] to grab all of the models for this project, and have inference done using ONNX Runtime[7]. This would allow me to run model inference on the client side, which would be so cool! The Transformers.js documentation made it seem like it would be easy enough, so I gave it a shot. The main python notebook (located at `project/model-exploration/text_to_music_onnx_conversion.ipynb` in the project repo[26]) use to convert Text-To-Music to an onnx format seemed like it was going well, just running into typical input/output shape issues. I made it as far as uploading it to HuggingFace[25] before I realized running inference on this model required a custom python library, *Samplings*[10]. For a few hours I tried to re-implement both the *TopPSampling* and *TemperatureSampling* methods before I decided to just use what the original author of the model had used, and go back to doing a full python implementation.

Successes

Basic-Pitch works great!

CREPE + Basic-Pitch make a mean combo

Limitations

Extensions

Conclusion - X words

In Conclusion...

If I were to do it again

If I were to start this project over, I believe my approach would change considerably. I think I probably would've stuck with using something like gradio[17] or streamlit[13] to quickly make the frontend would've cut down my workload for getting a testable user interface, and allow me to put more time into prompt

tuning the models I chose, and maybe even fine tuning models that would work better to generate sight-reading passages.

There were a lot of really cool audio processing libraries that I wanted to try, such as Librosa[23], but didn't get around to because of my approach. I believe that there are lots of signal processing implementations for audio analysis that I could've utilized if I wasn't so focused on using machine learning models for every aspect of this project.

Further Work

Acknowledgements

- I'm grateful to my sister Anna, and my partner J for proof reading through this report too many times to count, and supporting me during stressful times.
- Pandoc was used to generate this report from a markdown file
- Zotero was used to manage sources, and generate a BibTex file for references/citations
- draw.io was used for all of the Sight Reader Pro mockups
- <https://plantuml.com/> was used for creating my architecture diagrams

References

- [1] CREPE Interactive Demo. <https://marl.github.io/crepe/>.
- [2] Docker | Secure Sandboxes for AI Agents.
- [3] 2024. Empowering Students through Sight Reading. <https://nafme.org/>.
- [4] 2026. Huggingface/transformers.js. Hugging Face.
- [5] MLflow - Open Source AI Platform for Agents, LLMs & Models. *MLflow AI Platform*. <https://mlflow.org>.
- [6] NumPy. <https://numpy.org/>.
- [7] ONNX Runtime. <https://onnxruntime.ai/>.
- [8] React. <https://react.dev/>.
- [9] React Bootstrap | React Bootstrap. <https://react-bootstrap.netlify.app/>.
- [10] 2022. Samplings. *PyPI*. <https://pypi.org/project/samplings/>.
- [11] Sight Reading Factory®. <https://www.sightreadingfactory.com>.
- [12] Sight Reading Trainer. <https://sightreading.training/>.
- [13] 2021. Streamlit • A faster way to build and share data apps. <https://streamlit.io/>.
- [14] TensorFlow.js | Machine Learning for JavaScript Developers. *TensorFlow*. <https://www.tensorflow.org/js>.
- [15] Vite. *vitejs*. <https://vite.dev>.
- [16] 2026. Web MIDI API | MDN. *MDN Web Docs*. https://developer.mozilla.org/en-US/docs/Web/API/Web_MIDI_API.
- [17] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. and Zou, J. 2019. Gradio: Hassle-free sharing and testing of ML models in the wild.
- [18] Bittner, R.M., Bosch, J.J., Rubinstein, D., Meseguer-Brocal, G. and Ewert, S. 2022. A Lightweight Instrument-Agnostic Model for Polyphonic Note Transcription and Multipitch Estimation. arXiv.
- [19] Carnovalini, F., Espírito Santo, L. and Wiggins, G.A. 2025. Personalized Music Education: A Systematic Review of AI Generation Methods. *IEEE Access*. 13, (2025), 207433–207447. <https://doi.org/10.1109/ACCESS.2025.3639898>.
- [20] Copet, J., Kreuk, F., Gat, I., Remez, T., Kant, D., Synnaeve, G., Adi, Y. and Défossez, A. 2024. Simple and Controllable Music Generation. arXiv.

- [21] Hayward, C.M. and Gromko, J.E. 2009. Relationships among Music Sight-Reading and Technical Proficiency, Spatial Visualization, and Aural Discrimination. *Journal of Research in Music Education*. 57, 1 (2009), 26–36.
- [22] Kim, J.W., Salamon, J., Li, P. and Bello, J.P. 2018. CREPE: A Convolutional Representation for Pitch Estimation. arXiv.
- [23] McFee, B., Raffel, C., Liang, D., Ellis, D., McVicar, M., Battenberg, E. and Nieto, O. 2015. Librosa: Audio and Music Signal Analysis in Python. *Python in Science Conference* (Austin, Texas, 2015), 18–24.
- [24] Schreiber, H. and Müller, M. 2018. A Single-step Approach to Musical Tempo Estimation using a Convolutional Neural Network. *Proceedings of the 19th International Society for Music Information Retrieval Conference*. (Sep. 2018), 98–105. <https://doi.org/10.5281/zenodo.1492353>.
- [25] Thoraldson, T. Tthoraldson/text-to-music-onnxruntime · Hugging Face. <https://huggingface.co/tthoraldson/text-to-music-onnxruntime>.
- [26] Thoraldson, T. 2026. Tthoraldson/uol-final-project.
- [27] Wang, Y., Wu, S., Hu, J., Du, X., Peng, Y., Huang, Y., Fan, S., Li, X., Yu, F. and Sun, M. 2025. NotaGen: Advancing Musicality in Symbolic Music Generation with Large Language Model Training Paradigms. arXiv.